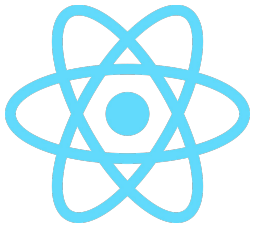
Universidad de La Laguna

React

Course 2025-2026

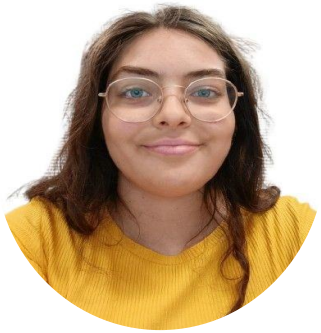
React



Link to code examples in these slides

Team

Candela García Cruz
candela.cruz.13@ull.edu.es



Sara Darias Sánchez
sara.darias19@ull.edu.es

Sergio de la Barrera García
sergio.barrera.garcia.25@ull.edu.es

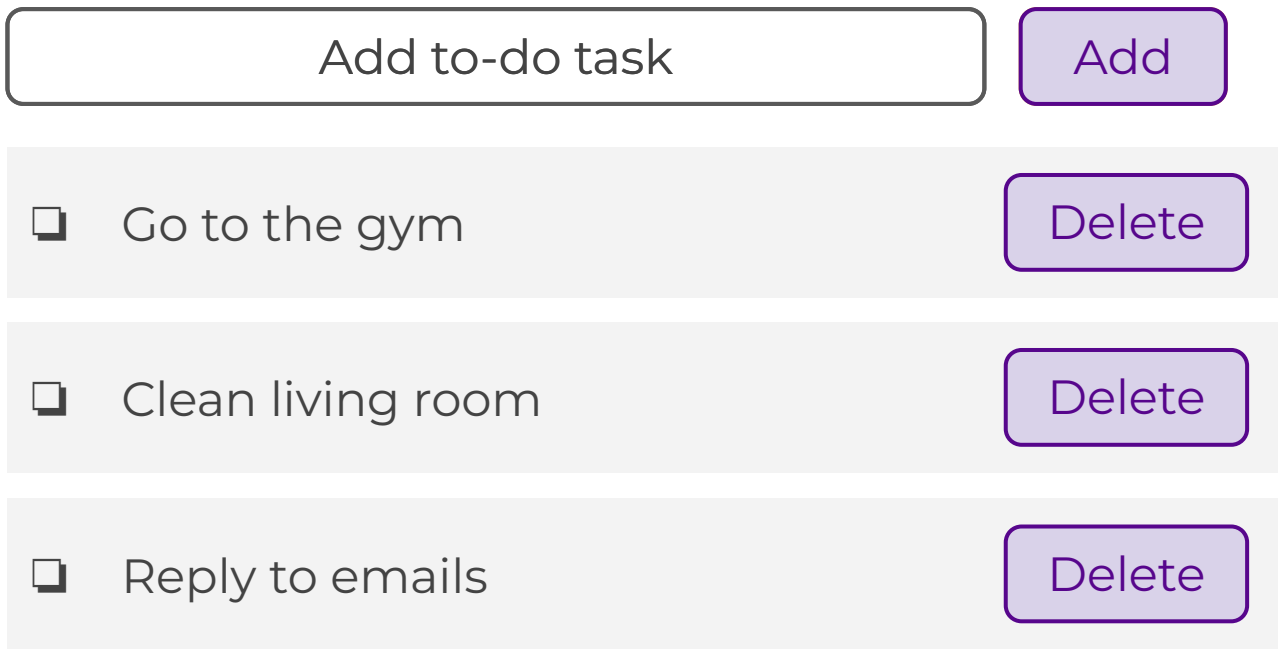
Index

Introduction to React

Why do frameworks exist?

Consider a to-do list creator:

- Render a list of tasks.
- Add a new task.
- Delete a task



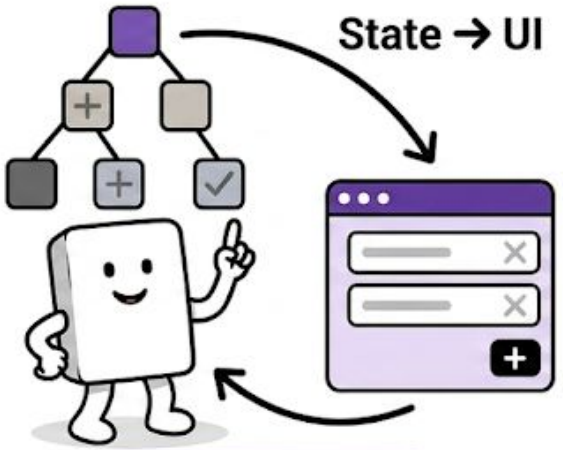
Why do frameworks exist? (II)

Consider a to-do list creator:

- Render a list of tasks.
- Add a new task.
- Delete a task

Each of our goals is theoretically simple in isolation.

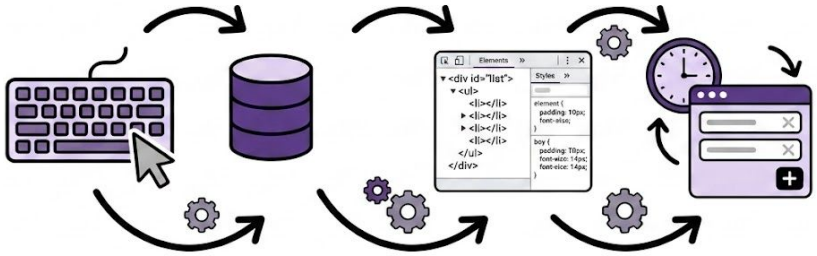
But... **every time we change our application's state, we need to update the user interface to match.**



Why do frameworks exist? (III)

Without a framework:

- Add a new task.
  1. Detect the user input.
  2. Update the data.
  3. Manually update the DOM.
  4. Refresh related elements (counter, list, etc.)

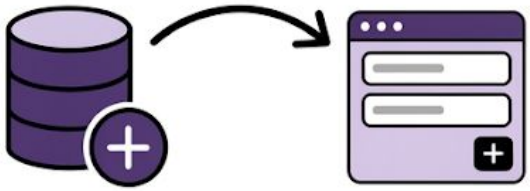


Every state change requires manual interface updates.

- More features (more code).
- More DOM manipulation.
- **More complexity and possible errors.**

Why do frameworks exist? (IIII)

With a framework:



- Add a new task.
  1. Update the state.
  2. The framework updates the DOM automatically.

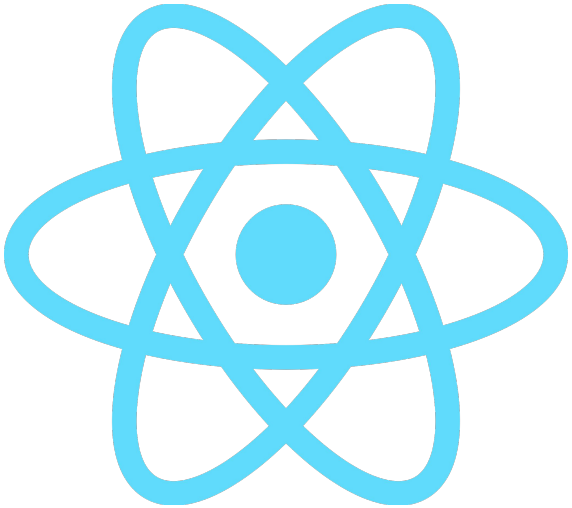
Frameworks provide a better developer experience.

They don't bring brand-new powers to TypeScript...

...they give you easier access to TypeScript powers so you can develop web applications more effectively in day-to-day work.

React

- Open-source JavaScript library created by Facebook in 2011.
- Based on reusable components.
- Follows a declarative approach.
- Uses a **virtual DOM** to efficiently update the user interface.
- Uses **TSX** syntax to write HTML-like code inside TypeScript (more readable).



Which framework is the best?

React (II)

React setup

```
npm create vite@latest [your project name]
```

React setup (II)

```
> npx
> create-vite trabajo-react

◆  Select a framework:
   ○ Vanilla
   ○ Vue
   ● React
   ○ Preact
   ○ Lit
   ○ Svelte
   ○ Solid
   ○ Qwik
   ○ Angular
   ○ Others
```

React setup (III)

```
> npx
> create-vite trabajo-react

|
◇  Select a framework:
|  React
|
◆  Select a variant:
|  ● TypeScript
|  ○ TypeScript + SWC
|  ○ JavaScript
|  ○ JavaScript + SWC
|  ○ React Router v7 ↗
|  ○ TanStack Router ↗
└
```

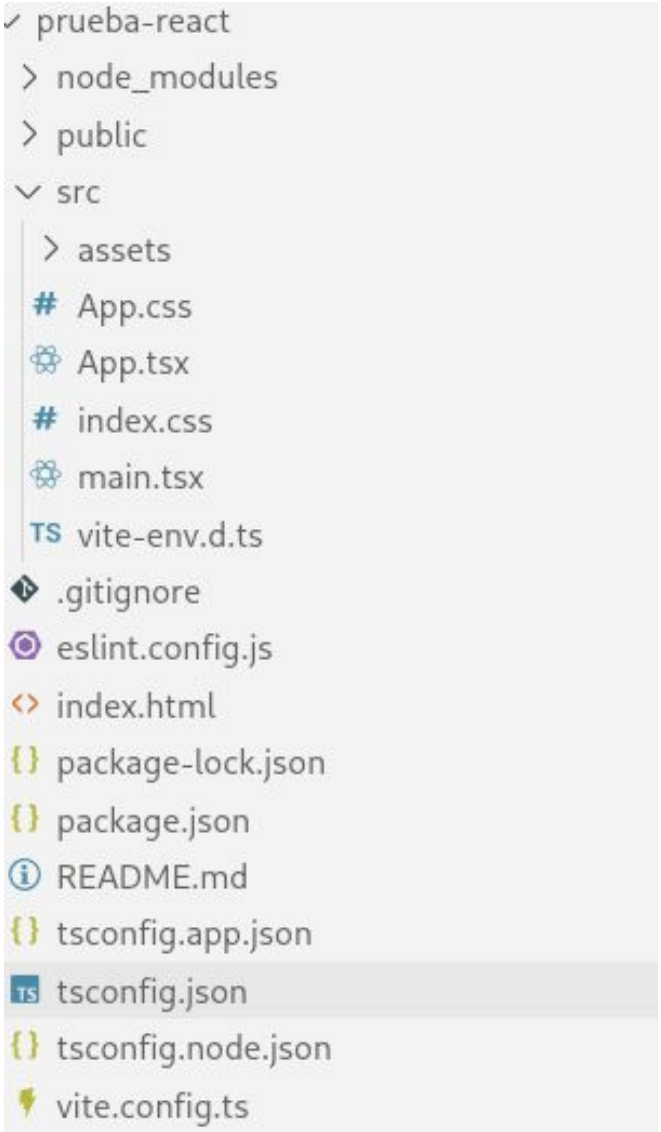
React setup (IV)

```
> npx
> create-vite trabajo-react

|
◇  Select a framework:
|  React
|
◇  Select a variant:
|  TypeScript
|
◇  Scaffolding project in /home/usuario/Trabajo/trabajo-react...
|
└  Done. Now run:

  cd trabajo-react
  npm install
  npm run dev
```

React setup (V)



React implementations

Hello world

- React uses a container to render HTML in a web page.
- Typically, this container is a element in the index.html file.

```html
<!doctype html>
<html lang="en">
  <head>
    <title>2025-2026-pai-react</title>
    <script type="module" src="/src/main.tsx"></script>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

index.html

Hello world (II)

- createRoot is a built-in function used to create a root node for a React application.

  Defines the HTML element where a React component should be displayed.

- The render method defines what to render in the HTML container.

```
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

main.tsx

Hello world (II)

- Content that is displayed in the container:

```tsx
import type { JSX } from 'react';

export function HelloWorld(): JSX.Element {
  return (
    <div className="App">
      <h1>Hello World!</h1>
    </div>
  );
}
```

hello-world.tsx

Events in React

Event handlers

What are Handlers?

your own functions that will be triggered in response to interactions

How to use them?

1. Define a function
2. Pass it as a prop to the appropriate JSX tag

Events in React (II)

Ways to define a handler

Inline

```jsx
function Button() {
  return (
    <button onClick={() =>
      alert('Button clicked!')}>
      Click me
    </button>
  );
}
```

Normal function

```jsx
function Button() {
  function handleHover() {
    console.log('Button
      hovered!');
  }

  return (
    <button
    onClick={handleHover}>
      Click me
    </button>
  );
}
```

events.tsx

Props

- Props are arguments passed into React components.
- Props are passed to components via HTML attributes.

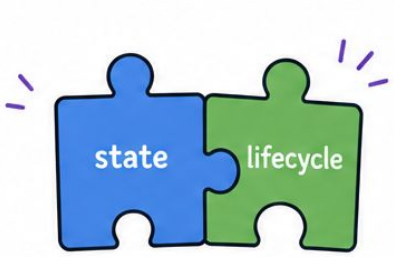
```ts
type BookProps = {
  title: string;
  author?: string;
  details?: {
    year: number;
    genre: string;
  };
  tags?: string[];
  extra?: string;
};
```

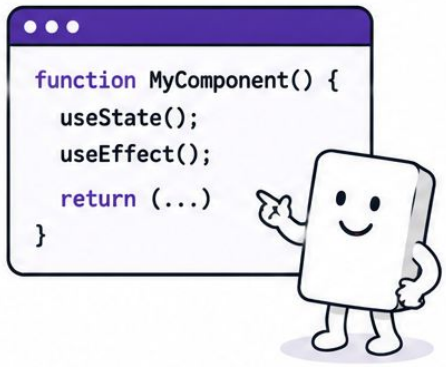
book-props.tsx

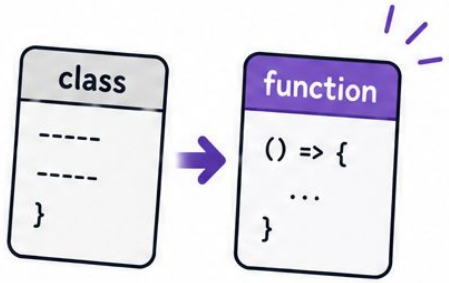
Hooks

What are Hooks?

- Functions that let you use state and lifecycle
- Work inside functional components
- Replace class-based components



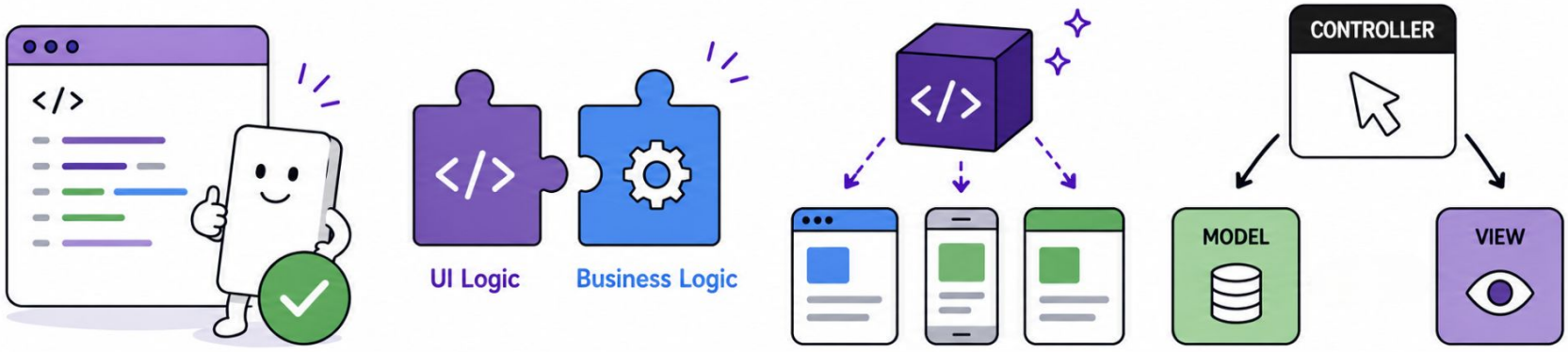




Hooks (II)

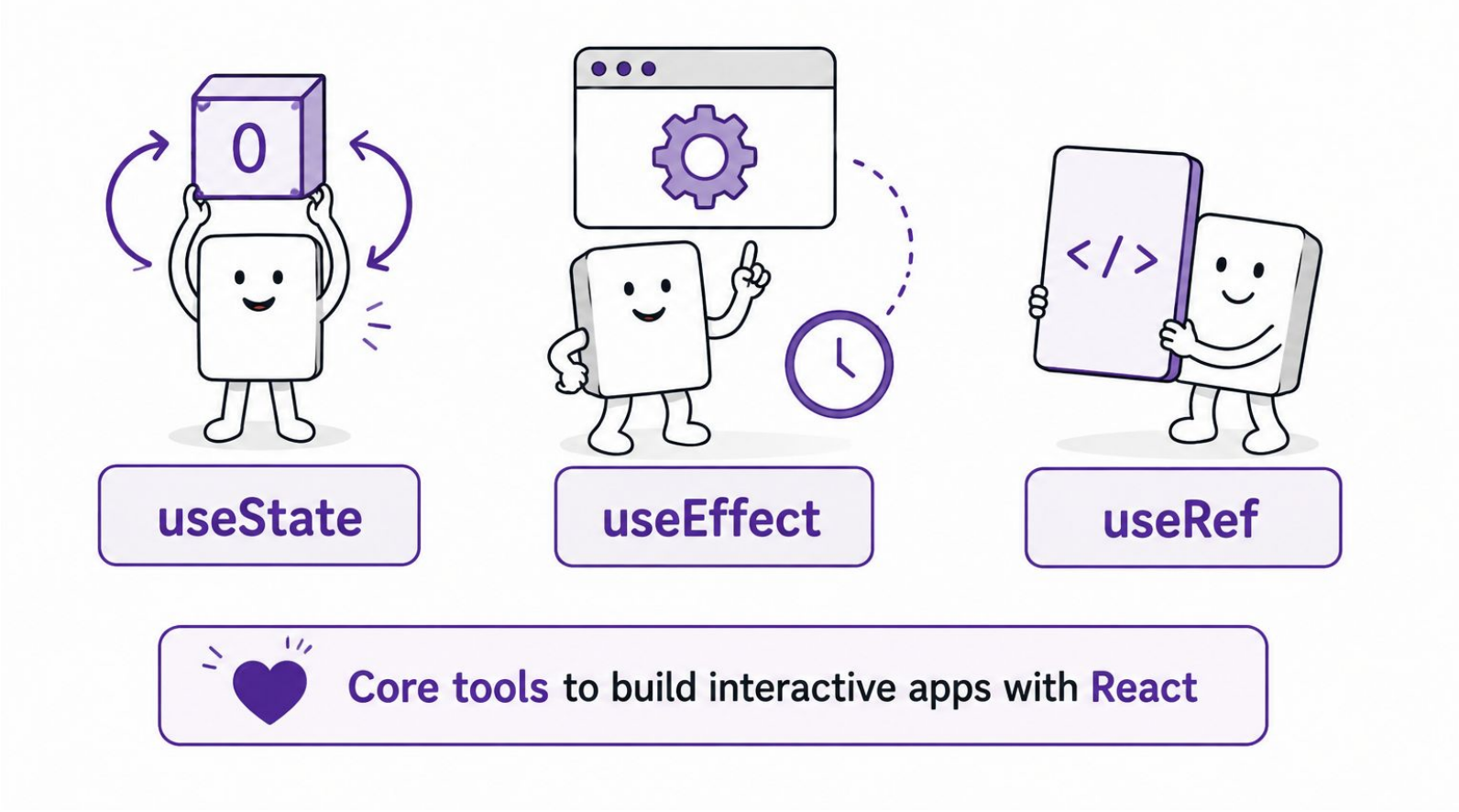
Why use Hooks?

- Cleaner and simpler code
- Better separation of logic
- Easier to reuse logic
- Fit perfectly with MVC in React



Hooks (III)

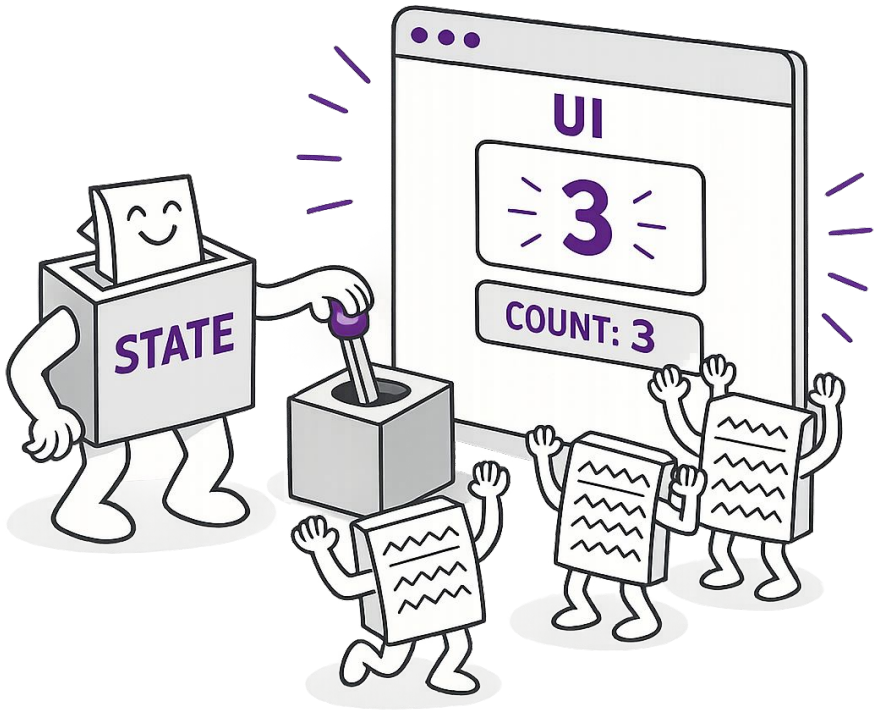
Most used Hooks



useState

State with useState

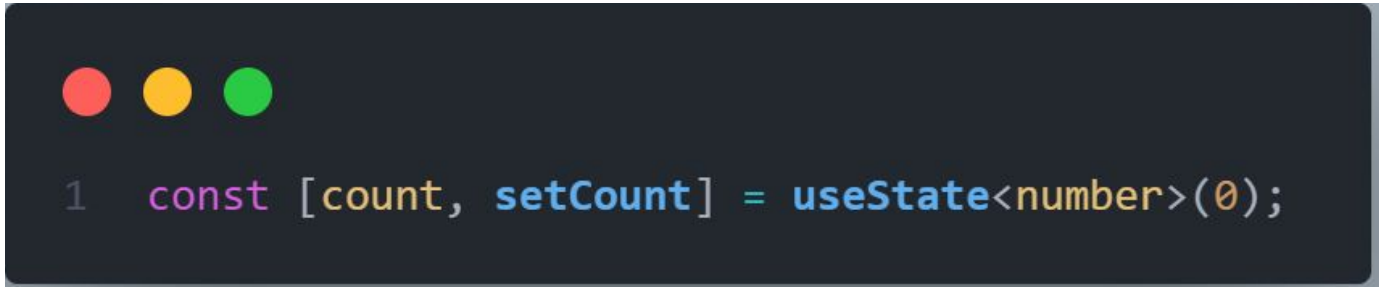
- Dynamic data
- Updates the UI automatically



useState (II)

useState Hook

- value + updater function



useState (III)

Interactive Example

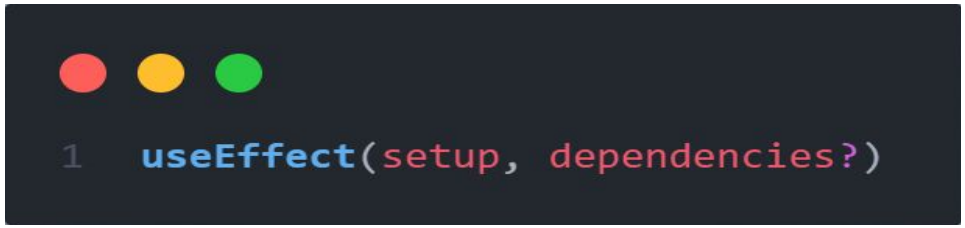
- Event → state change → UI update

```
1  <button onClick={(): void => setCount(count + 1)}>
2    Increment
3  </button>
```

use-state.tsx

UseEffect

Lets you synchronize a component with an external system.



two arguments

1. A setup function with setup code that connects to that system. It should return a cleanup function with cleanup code that disconnects from that system.
2. A list of dependencies including every value from your component used inside of those functions.

UseEffect (II)

Example

```
1   useEffect(() => {
2       if (count % 2 === 0) {
3         setColor('blue');
4       } else {
5         setColor('red');
6       }
7     }, [count]);
```

use-effect.tsx

UseRef

Lets you reference a value that’s not needed for rendering.

Parameters

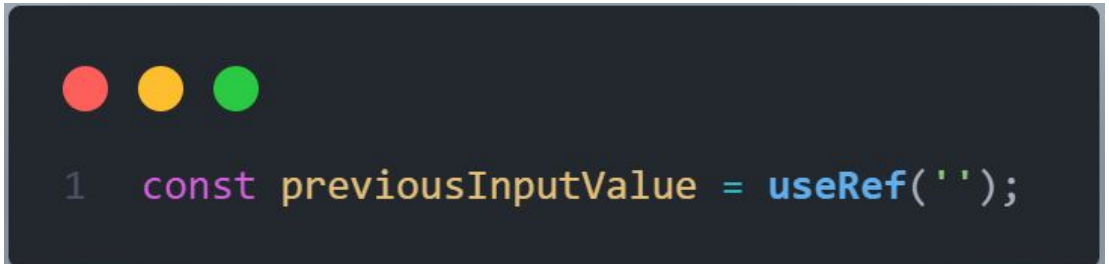
initialValue: The value you want the ref object’s current property to be initially.

Returns

a ref object with a single current property initially set to the initial value you provided.

UseRef (II)

Example

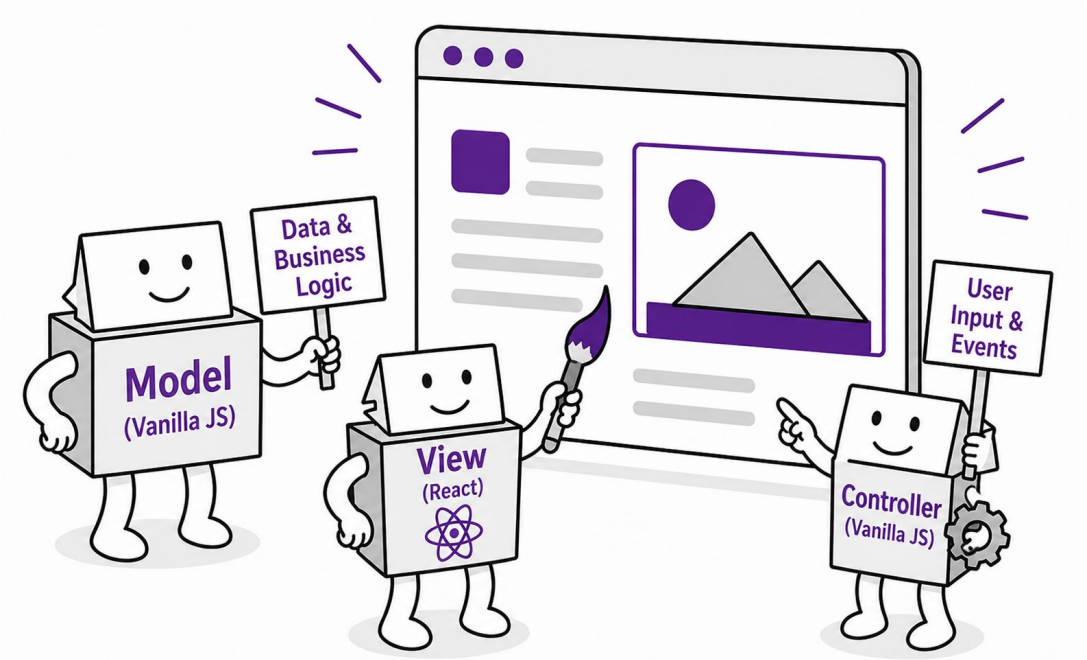


use-ref.tsx

MVC in React

Model <- Vanilla
View <- React
Controller <- Vanilla

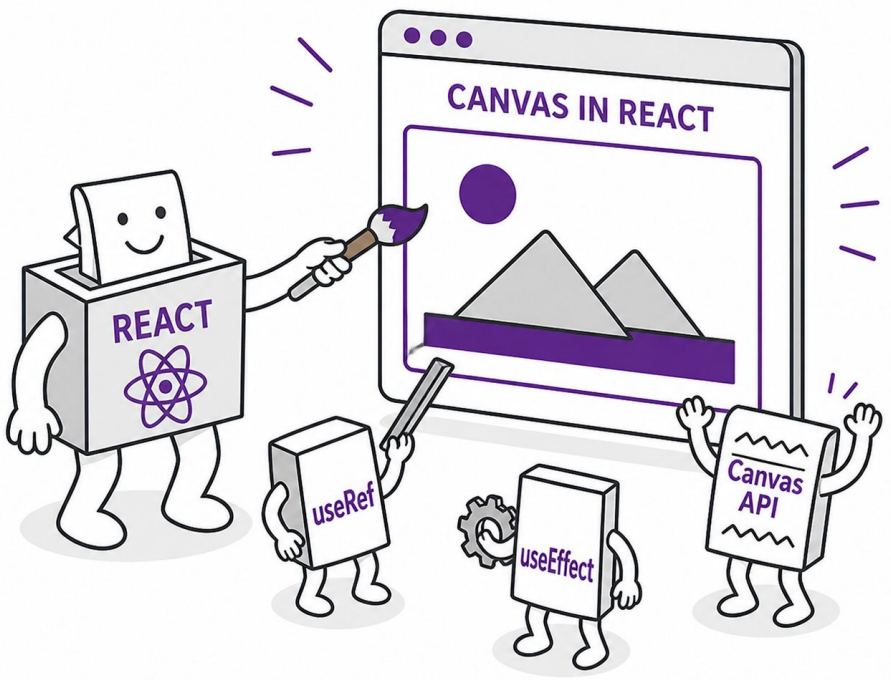
We need to have one component with everything



Canvas in React

Canvas remains the same

- HTML element for drawing graphics
- Useful for visual applications
- Drawing is done with React



Canvas in React (II)

Accessing Canvas

- useRef gives access to the canvas element

```
1  const canvasRef = useRef<HTMLCanvasElement | null>(null);
```

Canvas in React (III)

Drawing Canvas

- useEffect runs the drawing code

```
1  useEffect((): void => {
2    const canvas = canvasRef.current;
3    const context = canvas.getContext('2d');
4  }
```

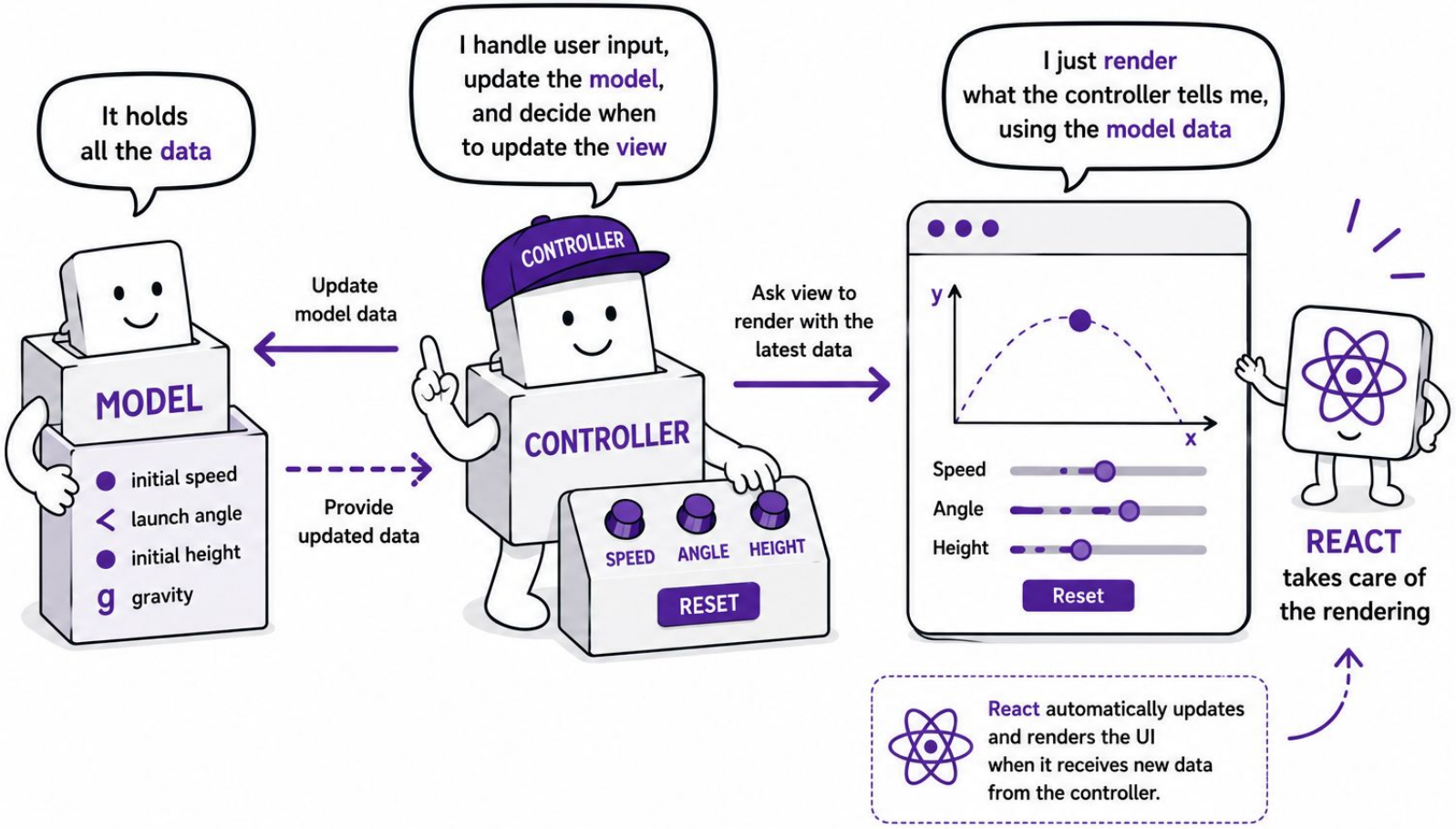
canvas-with-react.tsx

Final demo (Projectile)

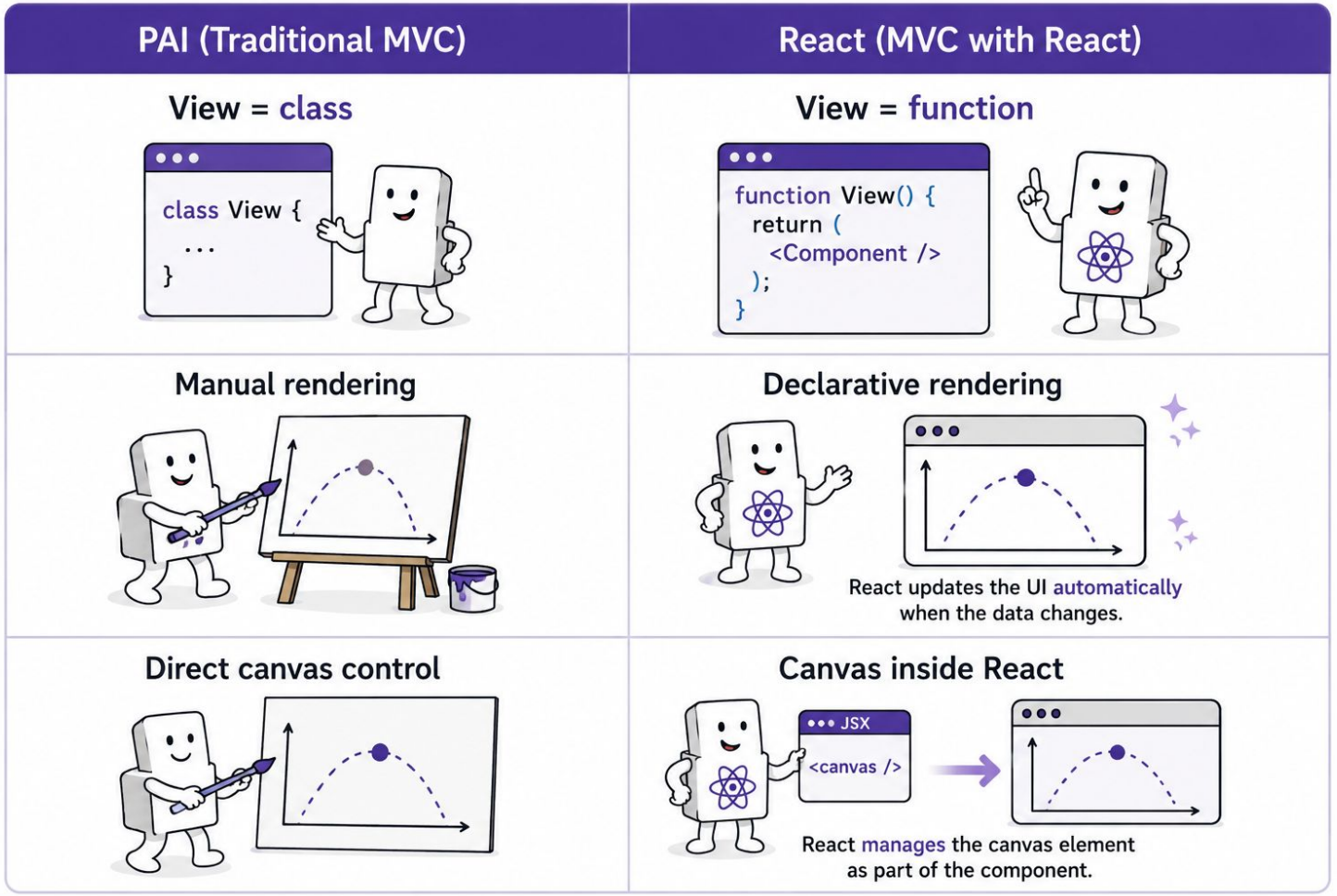
What we used? MVC

- Model → physics & data
- Controller → handles input & updates
- View → React + Canvas
- useEffect → drawing
- useRef → canvas access
- Controlled input

Final demo (Projectile II)



Final demo (Projectile III)

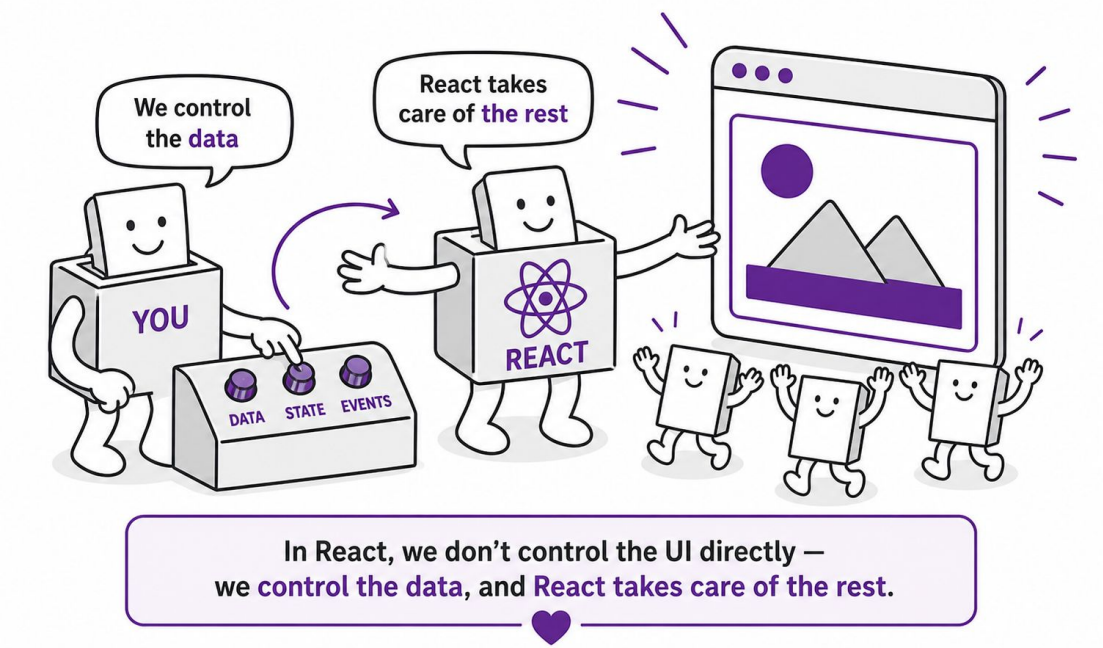


projectile.tsx

Conclusions

React Summary

- Components → structure the UI
- State → controls dynamic data
- Events → handle user interaction
- Canvas → enables custom graphics



Bibliography

- Why do frameworks exist?
- https://es.react.dev/reference/react
- https://adactio.medium.com/why-use-react-8ccd2ab4fea7
- https://react.dev/learn/build-a-react-app-from-scratch
- https://www.w3schools.com/REACT/default.asp
- https://react.dev/learn/responding-to-events
- https://react.dev/learn/state-a-components-memory
- https://react.dev/learn/passing-props-to-a-component
- https://react.dev/reference/react/useRef
- https://react.dev/reference/react/useEffect
- https://medium.com/@pdx.lucasm/canvas-with-react-js-32e133c05258
- https://medium.com/@martin.crabtree/react-creating-an-interactive-canvas-component-e8e88243baf6

Universidad de La Laguna

Any questions?